

RMC_RAW_Redirect_Spec

RMC - Reconfigurable DDR5 Memory Controller

RAW Redirect & Bypass Handler

Microarchitecture Specification

Status: Draft for Review

Scope: CIF domain - post Write Request Buffer allocation, pre Bank FSM Table scan

1. Purpose & Block Positioning

RAW Redirect is the Read-After-Write hazard resolution block within the CIF domain. Its job is to intercept every incoming read transaction before it reaches the Bank FSM Table and determine whether a pending write in the Write Request Buffer already holds the data that read needs. If it does, the data is served directly from the Write Data Buffer - no DRAM round-trip required.

This is a correctness block first, performance block second. Serving stale data from DRAM when a newer write is pending would be a silent data hazard - the kind that passes RTL simulation and fails only in directed hazard tests.

1.1 Position in the Pipeline

The block sits between two well-defined handoff points:

- **Input:** A read transaction that has just been allocated into the Read Request Buffer. At this point it has a valid `axi_tag`, translated address {BG, bank, row, col}, and `resp_slot` reserved in the Resp FIFO.
- **Output A (bypass path):** Data is routed directly into the Hold-and-Forward buffer, then emitted into the Resp FIFO - bypassing the Bank FSM Table, scheduler, and DRAM entirely.
- **Output B (normal path):** No match found - the read entry proceeds normally to the Bank FSM Table scan (Scheduler Stage 2).
- **Output C (stall path):** Partial overlap with gaps - the read is held in the Read Request Buffer with a STALL status flag. It is re-run through RAW Redirect each cycle until the conflicting write entry retires.

1.2 Read-Only Access

RAW Redirect has read-only access to:

- **Write Request Buffer** - all 32 valid entries, specifically: {valid, BG, bank, row, col, data_buf_idx, status, issue_cycle}

- **Write Data Buffer** - mask field only, indexed via data_buf_idx from the matched write entry.

RULE: RAW Redirect never modifies any write entry. The matched write proceeds through its full lifecycle (PENDING → ISSUED → DONE → WR_ACK) completely unaffected by a RAW hit.

2. Stage A - Binary CAM (Address Exact-Match)

Stage A performs a parallel exact-match search across all 32 Write Request Buffer entries. It runs every cycle for any read that is either newly allocated or currently stalled.

2.1 Match Key

The match key is the full translated address of the incoming read:

Field	Width	Source
BG	3b	Address Translator output, stored in Read Request Buffer
bank	2b	Address Translator output, stored in Read Request Buffer
row	17b	Address Translator output, stored in Read Request Buffer
col	10b	Address Translator output, stored in Read Request Buffer

Total match key width: 32 bits

Each of the 32 write entries is compared simultaneously against this key. An entry participates in the match only if its valid bit is set AND its status is PENDING or ISSUED (not DONE or ERROR - a DONE write has already been sent to DRAM and the data is no longer guaranteed to be in the Write Data Buffer).

NOTE: *Column is included in the match key. Two writes to the same row but different columns do not alias - they cover different 64-byte regions within the row. Stage A correctly treats them as independent entries.*

2.2 Output: row_match_vector

row_match_vector[31:0]

Each bit *i* is set if write entry *i* is valid, active (PENDING or ISSUED), and its {BG, bank, row, col} exactly equals the incoming read's translated address. Zero or more bits may be set simultaneously.

2.3 Implementation Primitive

Stage A uses XOR-based binary CAM cells - not subtractor-based magnitude comparators. This distinction is critical for RTL synthesis:

Primitive	Used where	How it works
XOR CAM cell	Stage A (RAW Redirect)	XOR all bits of key vs. stored value; NOR the result. Zero XOR = exact match. Single gate delay.
Subtractor comparator	Scheduler Stage 2 timing gates	global_cycle - next_legal_cycle; sign-bit check on bit[19]. Not a CAM.

These must not be conflated in the microarchitecture diagram or RTL. Stage A is a true CAM; Stage 2 is a magnitude comparator array.

2.4 Latency - Open Question

OPEN: Does Stage A add a pipeline register (adds one cycle to every read, including misses) or is it fully combinational and parallel with Read Request Buffer allocation? The answer determines whether the stall path holds for one cycle or zero cycles on a miss. This is unresolved and must be decided before RTL implementation. Recommendation: combinational parallel with allocation to avoid adding latency to the common case (no hazard).

3. Stage B - Mask Coverage Check

Stage B runs only when row_match_vector is non-zero. It determines whether the write(s) that hit in Stage A actually cover all the bytes that the read needs.

3.1 Multi-Hit Tiebreak (Newest Write Wins)

If row_match_vector has more than one bit set, Stage B must select exactly one write entry to work with before doing the mask check. The rule is: newest write wins.

Newest is defined by the issue_cycle timestamp stored in the Write Request Buffer at allocation time (global_cycle at the moment the write was allocated, not when it was issued to DRAM). This timestamp lives in the watermark manager's ts_reg[N-1:0].

Selection logic:

- winner = argmax(ts_reg[i] for i in hit_set)
- Implemented as a $\log_2(32) = 5$ -deep comparator tree. Not a CAM - this is pure priority encoding on timestamp values.
- Rationale: the newest write is the most up-to-date value for that address. Serving from an older write when a newer one is pending would give stale data.

NOTE: A duplicate-address scenario (two valid entries with identical {BG,bank,row,col}) should not arise under correct operation since the Scheduler would not issue a second write to the same address while the first is still PENDING. However, if it does occur, newest-wins is the safe tiebreak. 'Lowest index wins' is explicitly wrong here.

3.2 Mask Coverage Equation

Once the winning write entry is selected, the mask check is:

`coverable = (read_required_mask & write_valid_mask) == read_required_mask`

Signal	Width	Meaning
<code>read_required_mask</code>	64b	Byte-level mask of bytes the read needs. For a full BL16 non-masked read this is all-1.
<code>write_valid_mask</code>	64b	<code>mask[63:0]</code> from the Write Data Buffer entry pointed to by <code>data_buf_idx</code> of the winning write.
<code>coverable</code>	1b	1 = write covers all bytes the read needs. 0 = at least one needed byte is not covered.

4. Hit Classification and Resulting Actions

Four mutually exclusive cases, resolved in priority order:

4.1 Case 1 - No Match (`row_match_vector == 0`)

Stage A found no active write entry covering this read's address. Normal path: the read proceeds to Scheduler Stage 2 (Bank FSM Table scan). No intervention.

4.2 Case 2 - Full Hit (`write_valid_mask == all-1`)

The winning write covers every byte of the 64B region. Data is served entirely from the Write Data Buffer.

Actions:

- Read the full 512b payload from Write Data Buffer[`data_buf_idx`].
- Assemble a response packet: `resp_type=RD_DATA`, `tag=axi_tag`, `data=payload`, `status=OK`
- Route into Hold-and-Forward buffer (see §5).
- Mark the Read Request Buffer entry status = DONE.
- The write entry is NOT touched.

4.3 Case 3 - Partial Hit, Coverable (`row_match_vector != 0`, `coverable == 1`, `mask != all-1`)

The write covers all bytes the read needs, but the write itself is a masked (partial) write. Since every byte the read requires is present in the write buffer, this is still a clean bypass.

Actions: identical to Case 2. The covered bytes are pulled from the Write Data Buffer. No DRAM fetch.

4.4 Case 4 - Partial Hit With Gaps (row_match_vector != 0, coverable == 0)

The winning write exists and overlaps the read's address, but does not cover all bytes the read needs. Some bytes the read requires are absent from the write buffer - they are only in DRAM (or not yet written anywhere).

This is the dangerous case. Fetching from DRAM is incorrect here: DRAM still holds the pre-write value for the covered bytes. Serving those bytes from DRAM would give the read stale data for the bytes the write owns.

Action: STALL.

- Set Read Request Buffer entry status = STALL.
- Do not issue a DRAM read. Do not forward any data.
- Every cycle, re-run the Stage A + Stage B check for this entry.
- When the conflicting write entry reaches status = DONE (write has been issued to DRAM and acknowledged), re-run the check. If now coverable (Case 3) or no-match (Case 1, write is gone), proceed accordingly.

RULE: A stalled read must NOT degrade to a DRAM fetch. That would silently serve stale data for the overlapping bytes. The only safe exit from stall is: write retires → re-check.

5. Hold-and-Forward Buffer

There is exactly one write port into the Resp FIFO. Two independent response sources can fire in the same cycle: a RAW bypass hit and a DRAM read return. Without buffering, one would be dropped. The Hold-and-Forward buffer is a 2-deep register file that absorbs this collision.

5.1 Structure

Field	Width	Notes
hold_slot[0..1]	2 × full resp packet	Each slot holds one complete response: resp_type, tag, data, status
hold_valid[1:0]	2b	Per-slot occupancy bit

5.2 Arbitration and Drain Logic

Each cycle, the drain logic checks hold_valid and the Resp FIFO availability:

- If hold_valid[0] == 1 and Resp FIFO has a free slot: drain slot 0 → Resp FIFO, clear hold_valid[0].

- If `hold_valid[1] == 1` and Resp FIFO has a free slot and slot 0 was not just drained: drain slot 1 → Resp FIFO, clear `hold_valid[1]`.
- At most one drain per cycle (single Resp FIFO write port).

New arrivals from RAW bypass are written into the first available empty slot. The write-into-hold-slot and drain-from-hold-slot happen in the same cycle, so a new arrival may land in the slot just vacated by a drain.

5.3 Backpressure - RAW Bypass Gate

A RAW bypass can only proceed (Case 2 or Case 3) if at least one hold slot is free. If both slots are occupied (two responses queued but Resp FIFO is full), the bypass must not fire this cycle - the read stays as PENDING/STALL and retries next cycle.

```
raw_bypass_gate = (hold_valid != 2'b11) && gate_resp_fifo_avail
```

5.4 Third-Collision Case - Open Question

OPEN: *Can a third response arrive in the same cycle that both hold slots are occupied and the Resp FIFO is also full? The KB flags this as unresolved. Two sub-questions: (1) Is this structurally impossible given that `gate_resp_fifo_avail` blocks new DRAM reads from issuing? (2) If possible, does a 3rd slot suffice or is a credit counter needed? Left for friend to resolve with a rate-limiting proof or a structural argument.*

6. Signal Interface

All signals are in the CIF (AXI) clock domain. No CDC crossings inside RAW Redirect.

6.1 Inputs

Signal	Width	Source	Description
<code>rd_alloc_valid</code>	1b	Read Request Buffer	Pulsed high when a new read entry is allocated
<code>rd_bg[2:0]</code>	3b	Read Request Buffer	Bank group of incoming read
<code>rd_bank[1:0]</code>	2b	Read Request Buffer	Bank of incoming read
<code>rd_row[16:0]</code>	17b	Read Request Buffer	Row of incoming read
<code>rd_col[9:0]</code>	10b	Read Request	Column of incoming read

Signal	Width	Source	Description
		Buffer	
rd_axi_tag	varies	Read Request Buffer	AXI ID for response routing
rd_resp_slot[4:0]	5b	Read Request Buffer	Pre-reserved Resp FIFO slot index
rd_req_mask[63:0]	64b	Read Request Buffer	Byte mask of bytes this read requires
wr_valid[31:0]	32b	Write Request Buffer	Per-entry valid bits
wr_status[31:0][1:0]	64b	Write Request Buffer	Per-entry status (PENDING/ISSUED/DONE/ERROR)
wr_bg[31:0][2:0]	96b	Write Request Buffer	Per-entry bank group
wr_bank[31:0][1:0]	64b	Write Request Buffer	Per-entry bank
wr_row[31:0][16:0]	544b	Write Request Buffer	Per-entry row
wr_col[31:0][9:0]	320b	Write Request Buffer	Per-entry column
wr_data_idx[31:0][4:0]	160b	Write Request Buffer	Per-entry Write Data Buffer index
wr_ts[31:0][19:0]	640b	Write Request Buffer	Per-entry allocation timestamp (issue_cycle)
wdb_mask[31:0][63:0]	2048b	Write Data Buffer	Byte mask for all 32 entries (read-only)
wdb_data[31:0][511:0]	16384b	Write Data Buffer	Full 512b payload for all 32 entries (read-only)
hold_valid[1:0]	2b	Hold-and-Forward	Occupancy of hold slots
gate_resp_fifo_avail	1b	Resp FIFO	1 = Resp FIFO has a free unreserved slot

Signal	Width	Source	Description
stall_rd_valid[N]	1b/entry	Read Request Buffer	Asserted for each currently stalled read entry

6.2 Outputs

Signal	Width	Destination	Description
raw_hit	1b	Read Request Buffer	1 = bypass or stall decision made this cycle
raw_bypass_en	1b	Hold-and-Forward	1 = valid bypass data ready to write into hold slot
raw_bypass_data[511:0]	512b	Hold-and-Forward	Data payload for bypass response
raw_bypass_tag	varies	Hold-and-Forward	AXI tag for bypass response
raw_bypass_slot[4:0]	5b	Hold-and-Forward	Resp FIFO slot to target
raw_stall_en	1b	Read Request Buffer	1 = set this read entry's status to STALL
raw_no_hazard	1b	Scheduler Stage 2	1 = no RAW conflict, read may proceed to Bank FSM scan
row_match_vector[31:0]	32b	Internal (Stage B)	CAM hit vector - internal signal, not exported

7. Operational Flow - Cycle by Cycle

7.1 New Read Allocation (Normal Case - No Hazard)

- Cycle N: Read transaction allocated into Read Request Buffer. rd_alloc_valid asserted.
- Cycle N (combinational, or N+1 if pipelined - see §2.4 open question): Stage A evaluates row_match_vector. All bits zero.
- raw_no_hazard asserted. Read entry status remains PENDING.
- Cycle N+1 (or N+2): Read entry enters Scheduler Stage 2 scan normally.

7.2 New Read Allocation - Full RAW Hit

- Cycle N: rd_alloc_valid asserted. Stage A fires, row_match_vector has one or more bits set.
- Stage B runs: if multi-hit, argmax(ts_reg) selects winner. coverable evaluated.
- coverable == 1, mask == all-1 (or partial but covering): raw_bypass_en asserted.

- raw_bypass_gate checked: hold slot available and Resp FIFO has room. If yes: 512b data latched from Write Data Buffer, response assembled.
- Hold-and-Forward: bypass packet written into first free hold slot.
- Read Request Buffer entry status set to DONE.
- Write entry untouched - continues to DRAM normally.

7.3 New Read Allocation - Partial Hit With Gaps

- Cycle N: Stage A hits, Stage B evaluates coverable == 0.
- raw_stall_en asserted. Read Request Buffer entry status set to STALL.
- Read does NOT proceed to Scheduler Stage 2.
- Every subsequent cycle: RAW Redirect re-evaluates this stalled entry (stall_rd_valid asserted for it).
- Re-check Stage A: if conflicting write entry now has status == DONE, it no longer participates in the match (filtered out in Stage A). row_match_vector may now be zero → raw_no_hazard → proceed normally.
- If a newer write to the same address has arrived and is now coverable → bypass (Case 2/3).

7.4 DRAM Return + RAW Bypass Same Cycle

- Both raw_bypass_en and a DRAM read return assert in the same cycle.
- DRAM return goes to hold slot 0 (or whichever is free first - priority is implementation detail).
- RAW bypass goes to hold slot 1.
- hold_valid becomes 2'b11.
- Drain logic drains one slot per cycle on subsequent cycles as Resp FIFO frees up.

8. Correctness Invariants

These must hold at all times. Any implementation that violates these is architecturally broken regardless of timing closure:

#	Invariant
I-1	A write entry matched by a RAW hit is never retired early or modified by RAW Redirect. It proceeds through PENDING → ISSUED → DONE → WR_ACK independently.
I-2	A read with coverable == 0 must never be issued to DRAM while the conflicting write is still active (PENDING or ISSUED). Fetching from DRAM would return pre-write data for the covered bytes.
I-3	Newest-write-wins on multi-hit. Lowest-index-wins is incorrect and must not be implemented.

#	Invariant
I-4	A RAW bypass may only fire when raw_bypass_gate == 1 (free hold slot AND Resp FIFO has room). Firing without a free slot drops data.
I-5	Stage A only considers write entries with status == PENDING or ISSUED. A DONE write has completed its DRAM commit - it is no longer a hazard source and must not block a read.
I-6	Column is part of the match key. Writes to the same row but different columns are not hazards to each other.

9. Open Questions

#	Topic	Question	Impact
OQ-1	Stage A latency	Is Stage A combinational (parallel with RRB allocation) or pipelined (adds one cycle to every read)? Affects miss-path latency and stall exit timing.	ROB watermark, miss-path cycle count
OQ-2	3rd Hold-Forward collision	Can a 3rd response arrive when both hold slots are occupied? Is this rate-limited by construction (gate_resp_fifo_avail blocks DRAM reads when FIFO is full)? If not, does a 3rd slot suffice?	Hold-Forward buffer depth, backpressure design

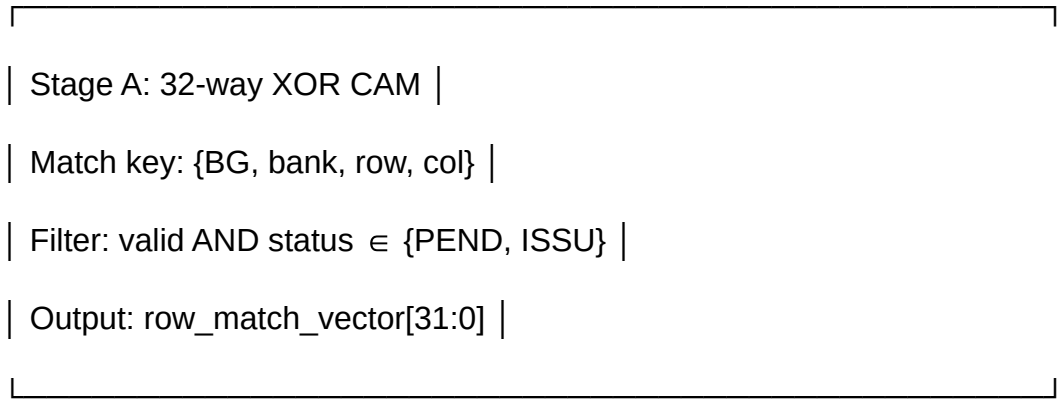
10. Block Summary

The following is the functional flow of RAW Redirect in text form:

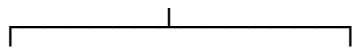
Read Request Buffer (new alloc or stalled entry)

|

v



|



all-0 non-zero

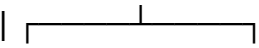
||

no hazard Stage B: Multi-hit tiebreak

| argmax(ts_reg) → winner entry

||

| Mask check: coverable?



| yes no

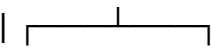
|||

| Bypass STALL

| path (re-check each cycle)

||

| raw_bypass_gate?



| yes no (hold slot full)

|||

| Write to retry next cycle

| Hold-Fwd

||

└─→ Proceed to

Scheduler Stage 2